Private User Data Aggregation

Verifiable Secure Aggregation for Federated Learning

Rust · BLS12-381 · Arkworks

Donval

EURECOM – Cryptography Project

February 2026

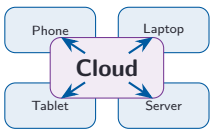
Outline

1. **Federated Learning**
   - Context and motivation
   - The secure aggregation problem
2. **PUDA Protocol**
   - Protocol overview
   - Threat model
3. **Technical Choices**
   - Rust and Arkworks
   - BLS12-381, Hash-to-$\mathbb{G}_1$, DLP Table
4. **The Four Phases**
   - Setup (Data Dealer)
   - EncTag (User)
   - Aggregate (Aggregator)
   - Verify (Analyzer)
5. **Performance Analysis**
   - Benchmarks and observations
6. **Conclusion**

Federated Learning



Why do we need secure aggregation?



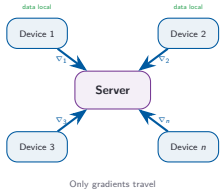
Data stays local, only **gradients** are shared.

But gradients can leak information!

What is Federated Learning?

Definition

Machine learning approach where the model is trained across **decentralized devices** holding local data, without exchanging raw data.



How it works:

- Data remains on device (privacy by design)
- Each device computes local gradients
- Only gradients are sent to central server
- Server aggregates: $\nabla_{\text{global}} = \frac{1}{n} \sum_i \nabla_i$
- Updated model is sent back to devices

The catch:

- Gradients can leak training data
- Server sees individual contributions

Applications:

- Google Keyboard
- Apple Siri
- Medical research
- Banking fraud detection

The Secure Aggregation Problem

Privacy Gap

Even without raw data, a **malicious server** can infer training data from gradients (gradient inversion attacks) and track individual contributions.

Existing approaches:

- **Differential Privacy**
  - \+ Provable guarantees
  - − Utility loss (noise added)
- **Secure MPC**
  - \+ Perfect security
  - − High communication cost
- **Homomorphic Encryption**
  - \+ Compute on ciphertexts
  - − Very slow in practice

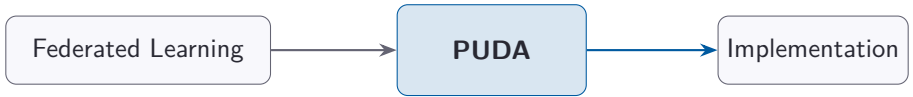
What we need

1. **Privacy**: Individual values hidden
2. **Accuracy**: Exact result (no noise)
3. **Verifiability**: Detect tampering
4. **Efficiency**: Practical performance

⇓

PUDA Protocol

PUDA Protocol



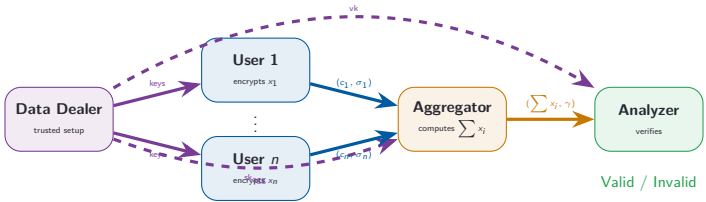
How does PUDA work?

PUDA = **P**rivate **U**ser **D**ata **A**ggregation

A cryptographic protocol that enables:

- Privacy-preserving aggregation using **mask cancellation**
- Public verifiability using **bilinear pairings**
- Exact results (no noise, no approximation)

PUDA: Protocol Overview



1. Setup
Key generation
(once)

2. EncTag
Encrypt + tag
(per user)

3. Aggregate
Compute sum
(mask cancellation)

4. Verify
Check integrity
(pairings)

Threat Model

Adversarial Model

- **Data Dealer**: Trusted (runs once)
- **Users**: Honest (follow protocol)
- **Aggregator**: Semi-honest
- **Analyzer**: Public verifier

Semi-honest Adversary $\mathcal{A}$

The aggregator:

- Executes protocol correctly
- Tries to learn individual x_i from the transcript $\{(c_i, \sigma_i)\}_{i \in [n]}$

Security Properties

Privacy:

- $\mathcal{A}$ cannot distinguish (x_i) from (x_i') with same aggregate
- Reduces to CDH in $\mathbb{G}_1$

Verifiability:

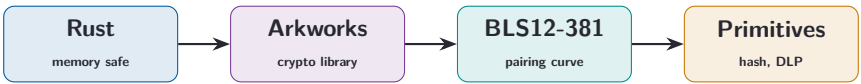
- $\mathcal{A}$ cannot forge valid $(\tilde{X}, \tilde{\gamma})$ with $\tilde{X} \neq \sum x_i$
- Reduces to co-CDH

Correctness:

- Honest execution yields $X_k = \sum_i x_{i,k}$

Technical Choices





Rust and Arkworks

Language: Rust

- Memory safety without garbage collector
- Zero-cost abstractions
- Strong type system
- No undefined behavior

Why it matters for crypto:

- Constant-time operations possible
- Memory zeroization (secrets don't leak)
- No implicit copies of sensitive data
- Compiler catches many bugs

Library: Arkworks

ark-bls12-381:

- Pairing-friendly elliptic curve
- 128-bit security level
- Optimized implementation

ark-ec, ark-ff:

- Elliptic curve arithmetic
- Finite field operations
- Serialization utilities

Ecosystem

Same libraries used by:
Zcash, Aleo, Mina Protocol

BLS12-381: Algebraic Structure

Three Groups

Cyclic groups of prime order $r \approx 2^{255}$:

$$\mathbb{G}_1 \subset E(\mathbb{F}_p) \qquad \text{(48 bytes compressed)}$$
$$\mathbb{G}_2 \subset E'(\mathbb{F}_{p^2}) \qquad \text{(96 bytes compressed)}$$
$$\mathbb{G}_T \subset \mathbb{F}^*_{p^{12}} \qquad \text{(target group)}$$

Bilinear Pairing:

$$e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$$

Key property (bilinearity):

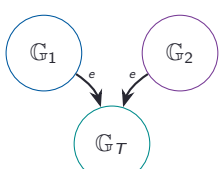
$$\boxed{e([a]P, [b]Q) = e(P, Q)^{ab}}$$

Why pairings?

Enable verification without secrets:

$$e(\gamma, P_2) \stackrel{?}{=} \prod_k e(V_k, \mathsf{vk}_k)$$

- Check $\mathbb{G}_1$-$\mathbb{G}_2$ relationships
- Without revealing scalar values
- Anyone can verify (public)



Hash-to-$\mathbb{G}_1$: Mapping Strings to Curve Points

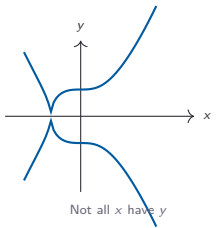
The Problem

Need function $\mathcal{H} : \{0,1\}^* \to \mathbb{G}_1$

Used for: computing $\mathcal{H}(\tau \| k)$ in encryption

Challenge:

- Curve equation: $y^2 = x^3 + 4$
- Not every $x \in \mathbb{F}_p$ yields valid y
- Only $\sim 50\%$ of x values work



Solution: Try-and-Increment

1. Hash input with counter
2. Interpret as x-coordinate
3. Check if $y^2 = x^3 + 4$ is square
4. If yes: compute y, return point
5. If no: increment counter, retry

Important details:

- Uses SHA-512 for hashing
- Legendre symbol to check quadratic residue
- Cofactor multiplication for subgroup
- Expected iterations: ~ 2

Hash-to-$\mathbb{G}_1$: Implementation

Key insight: We hash, check if valid curve point exists, multiply by cofactor to land in $\mathbb{G}_1$.

```
pub fn map_to_group(message: &str) -> G1 {
    let mut counter = 0u64;
    loop {
        // 1. Hash message with counter to get x-coordinate candidate
        let input_str = format!("{:0width$b}{}", counter, message, width = max_iterations);
        let (x, sign_selector) = h_prime(input_str.as_bytes());  // Uses SHA-512

        // 2. Compute f(x) = x^3 + ax + b (curve equation)
        let f_x = x.pow([3]) + coeff_a * x + coeff_b;

        // 3. Check if f(x) is a quadratic residue (Legendre symbol)
        if f_x.legendre().is_qr() {
            let root = f_x.sqrt().unwrap();          // Compute y = sqrt(f(x))
            let point = G1Affine::new_unchecked(x, root);

            // 4. Multiply by cofactor to ensure point is in G1 subgroup
            let final_point = point.into_group().mul_bigint(cofactor_big);
            if !final_point.is_zero() { return final_point; }
        }
        counter += 1;  // Try next candidate
    }
}
```

DLP Table: Discrete Log Recovery

Problem: After mask cancellation, we have $V = [X]P_1$. How to recover integer X?

Solution: Precomputed Table

Build lookup table at startup:

$$T = \{[i]P_1 \mapsto i\}_{i=0}^{65535}$$

Query: serialize V, lookup in HashMap
Complexity: $O(1)$ per lookup

Constraint

$n \times \max(x_i) < 65536$
With $n = 100$: $\max \leq 655$

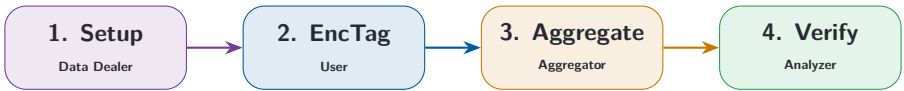
```rust
const DLP_TABLE_SIZE: u64 = 65536;

fn init_dlp_lookup_table() -> HashMap<Vec<u8>, u64> {
    let g1_gen = G1::generator();
    let mut table = HashMap::new();
    let mut current = G1::zero();  // Start at identity

    for i in 0..DLP_TABLE_SIZE {
        let mut bytes = Vec::new();
        current.serialize_compressed(&mut bytes);
        table.insert(bytes, i);  // Map point -> scalar
        current += g1_gen;       // Next: (i+1)*G
    }
    table
}

pub fn find_discrete_log(target: G1) -> u64 {
    let mut bytes = Vec::new();
    target.serialize_compressed(&mut bytes);
    *DLP_TABLE.get(&bytes).unwrap()  // O(1) lookup
}
```

The Four Phases



Keys
↓
All actors

$\mathsf{sk}_{\mathsf{agg}} = -\sum \mathsf{ek}_i$

(c_i, σ_i)
↓
Aggregator

masked ciphertexts

$(\sum x_i, \gamma)$
↓
Analyzer

masks cancelled

Valid/Invalid
↓
Result

pairing check

Phase 1: Setup (Data Dealer)

Input

- n: number of users
- κ: vector dimension

Output

- $\text{pp} = (P_1, P_2)$: generators
- $\{(\text{ek}_i, \text{tk}_i)\}$: user keys
- sk_{agg}: aggregator key
- vk: verification key

Algorithm

1. Sample generators P_1, P_2
2. Sample tag coefficients $\mathbf{a} \xleftarrow{\$} \mathbb{F}_r^{\kappa}$
3. For each user i:
   - $\text{ek}_i \xleftarrow{\$} \mathbb{F}_r$ (encryption key)
   - $b_i \xleftarrow{\$} \mathbb{F}_r$ (tag component)
4. Compute $\text{sk}_{\text{agg}} = -\sum_i \text{ek}_i$
5. Compute vk_a, vk_b in $\mathbb{G}_2$

Key constraint:

$$\boxed{\text{sk}_{\text{agg}} = -\sum_{i=1}^{n} \text{ek}_i}$$

Phase 1: Setup – Code

Key insight: We accumulate all ek_i into ak, then negate it for the aggregator key.

```rust
pub fn setup(num_users: usize, rng: &mut StdRng, kappa: usize)
    -> (PublicParams, Vec<UserSecretKey>, AggregatorSecretKey, DataDealerVK)
{
    let p_1 = G1::generator();
    let p_2 = G2::generator();

    // Tag coefficients: a = (a_1, ..., a_kappa) sampled uniformly
    let a: Vec<ScalarField> = (0..kappa).map(|_| ScalarField::rand(rng)).collect();
    let vk_a: Vec<G2> = a.iter().map(|a_val| p_2 * a_val).collect();  // vk_a[k] = a[k] * P2

    let mut ak: ScalarField = ScalarField::zero();  // Will accumulate sum of ek_i

    for i in 0..num_users {
        let ek_i = ScalarField::rand(rng);  // Random encryption key
        ak += ek_i;                          // Accumulate for aggregator key
        let b_i = ScalarField::rand(rng);    // Random tag component
        vk_b[i] = p_2 * b_i;                 // vk_b[i] = b_i * P2
        user_keys.push(UserSecretKey { id: i, ek: ek_i, tk: (a.clone(), b_i) });
    }

    let aggregator_key = AggregatorSecretKey { ska: -ak };  // NEGATION enables mask cancellation
    // ...
}
```

Phase 2: EncTag (User)

Input

- $\mathbf{x}_i = (x_{i,1}, \ldots, x_{i,\kappa})$: data
- τ: session identifier
- $(\mathsf{ek}_i, \mathsf{tk}_i)$: user keys

Output

- $\mathbf{c}_i \in \mathbb{G}_1^{\kappa}$: ciphertexts
- $\sigma_i \in \mathbb{G}_1$: authentication tag

Privacy: The mask $\mathsf{ek}_i \cdot \mathcal{H}(\tau \| k)$ hides $x_{i,k}$ under CDH assumption.

Formulas

Ciphertext (for each k):

$$c_{i,k} = \underbrace{[x_{i,k}]P_1}_{\text{data}} + \underbrace{[\mathsf{ek}_i] \cdot \mathcal{H}(\tau \| k)}_{\text{mask}}$$

Tag:

$$\sigma_i = \underbrace{[b_i] \cdot \mathcal{H}(\tau)}_{\text{user binding}} + \underbrace{\sum_k [a_k x_{i,k}]P_1}_{\text{data commitment}}$$

The tag commits to all values while binding to user identity via b_i.

Phase 2: EncTag – Code

Key insight: Each ciphertext $c[k]$ combines encoded data with a mask. The tag commits to all data.

```
pub fn enc_tag(uk: &UserSecretKey, x_i: Vec<u64>, tau: u64,
               kappa: usize, pp: &PublicParams) -> CiphertextAndTag
{
    // 1. Compute ciphertexts: c[k] = x[k]*P1 + ek*H(tau||k)
    let mut c: Vec<G1> = vec![G1::zero(); kappa];
    for k in 0..kappa {
        let h = map_to_group(&format!("{}{}", tau, k));  // H(tau||k)
        c[k] = pp.p_1 * ScalarField::from(x_i[k])         // Data: x[k] * P1
             + h * uk.ek;                                  // Mask: ek * H(tau||k)
    }

    // 2. Compute tag: sigma = b*H(tau) + Sum(a[k] * x[k] * P1)
    let h_tau = map_to_group(&tau.to_string());
    let mut sigma = h_tau * uk.tk.1;  // User binding: b_i * H(tau)

    for (k, x_val) in x_i.iter().enumerate() {
        let a_k = uk.tk.0[k];
        sigma += pp.p_1 * (ScalarField::from(*x_val) * a_k);  // Data commitment
    }

    CiphertextAndTag { c, sigma }
}
```

Phase 3: Aggregate (Aggregator)

Input

- $\{(\mathbf{c}_i, \sigma_i)\}_{i\in[n]}$: all ciphertexts
- $\mathsf{sk}_{\mathsf{agg}}$: aggregator key
- τ: session identifier

Output

- $\mathbf{X}$ where $X_k = \sum_i x_{i,k}$
- $\gamma = \sum_i \sigma_i$

Privacy: Aggregator learns $\sum x_i$ but **not** individual x_i values!

Mask Cancellation

For each k:

$$V_k = \sum_i c_{i,k} + [\mathsf{sk}_{\mathsf{agg}}] \cdot \mathcal{H}(\tau \| k)$$

$$= \sum_i [x_{i,k}]P_1 + \underbrace{[\sum_i \mathsf{ek}_i + \mathsf{sk}_{\mathsf{agg}}] \cdot \mathcal{H}}_{=0}$$

$$= [X_k]P_1$$

Then: $X_k \leftarrow \mathrm{DLP_lookup}(V_k)$

Phase 3: Aggregate – Code

Key insight: Adding $\mathsf{sk}_{\mathsf{agg}} \cdot H$ cancels all the $\mathsf{ek}_i \cdot H$ masks, revealing $[X_k]P_1$.

```
pub fn aggregate(agg_sk: &AggregatorSecretKey, inputs: &[CiphertextAndTag],
                 tau: u64, kappa: usize, _pp: &PublicParams) -> (Vec<u64>, G1)
{
    let mut x: Vec<u64> = Vec::new();

    for k in 0..kappa {
        let h = map_to_group(&format!("{}{}", tau, k));  // H(tau||k)

        // Sum all ciphertexts for dimension k
        let mut v = G1::zero();
        for ct in inputs {
            v += ct.c[k];  // v = sum of c_i[k]
        }

        v += h * agg_sk.ska;  // ADD aggregator's mask: CANCELS all user masks!

        x.push(find_discrete_log(v));  // Recover X_k from [X_k]P1
    }

    // Combine all tags
    let mut gamma = G1::zero();
    for ct in inputs { gamma += ct.sigma; }

    (x, gamma)
}
```

Phase 4: Verify (Analyzer)

Input

- $(X_1, \ldots, X_\kappa)$: claimed aggregate
- $\gamma \in \mathbb{G}_1$: combined tag
- $\mathsf{vk} = (\mathsf{vk}_a, \mathsf{vk}_b)$
- τ: session identifier

Output

- accept: result is correct
- reject: tampering detected

Public verification: No secrets needed!

Pairing Equation

$$e(\gamma, P_2) \stackrel{?}{=} \prod_k e([X_k]P_1, \mathsf{vk}_{a,k}) \cdot \prod_i e(H_\tau, \mathsf{vk}_{b,i})$$

Why it works:

- Left side: pairing of combined tag
- Right side: reconstruction from X_k and verification keys
- By bilinearity, equal iff consistent

Phase 4: Verify – Code

Key insight: We compute both sides of the pairing equation and compare. If equal, the aggregation is correct.

```
pub fn verify(data_vk: &DataDealerVK, gamma: G1, x_tau: Vec<u64>,
              tau: u64, kappa: usize, pp: &PublicParams) -> bool
{
    // LEFT SIDE: e(gamma, P2)
    let lhs = Bls12_381::pairing(gamma, pp.p_2).0;

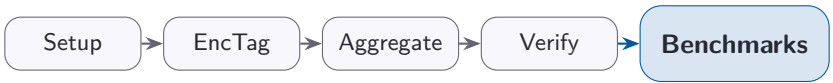
    // RIGHT SIDE: product of pairings
    let mut rhs = TargetField::one();

    // Part 1: for each dimension k, compute e(X_k * P1, vk_a[k])
    for k in 0..kappa {
        rhs *= Bls12_381::pairing(
            pp.p_1 * ScalarField::from(x_tau[k]),  // X_k * P1
            data_vk.vk_a[k]                         // a_k * P2
        ).0;
    }

    // Part 2: for each user i, compute e(H(tau), vk_b[i])
    let h_tau = map_to_group(&tau.to_string());
    for i in 0..data_vk.vk_b.len() {
        rhs *= Bls12_381::pairing(h_tau, data_vk.vk_b[i]).0;
    }

    lhs == rhs  // True if aggregation is correct
}
```

Performance Analysis



Real measurements on my machine

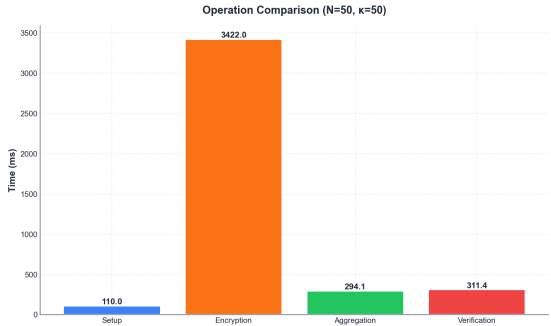
Configuration: 5 trials average, $\max(x_i) = 650$, single-threaded

Benchmark Results

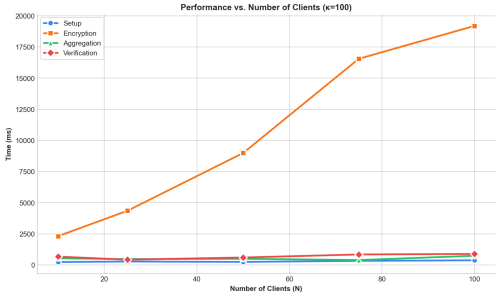
(n, κ)	Setup	Enc	Agg	Ver
25, 25	55	641	173	125
50, 50	110	3422	294	311
75, 75	336	12000	495	857
100, 100	368	19189	734	878

All times in milliseconds. Encryption is total for all n users.

Key finding: Encryption takes **90%+** of total time at scale. But it's parallelizable across users!

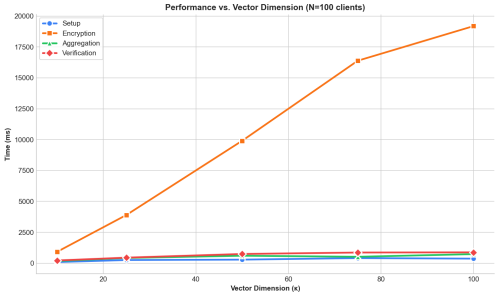


Scaling with Users and Dimensions



Varying n (users):

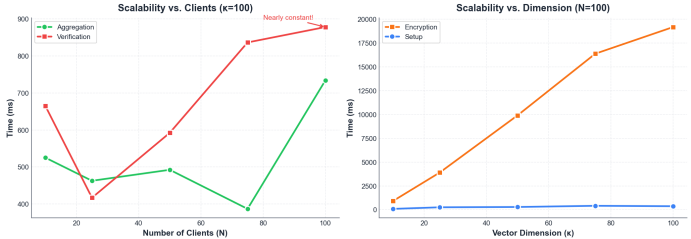
- Encryption scales linearly (n users)
- Aggregation nearly constant
- Verification linear in n



Varying κ (dimensions):

- All phases scale with κ
- Encryption per user: $O(\kappa)$
- Hash-to-curve dominates cost

Scalability Analysis



Complexity:

- Setup: $O(n + \kappa)$ in $\mathbb{G}_2$ multiplications
- EncTag: $O(\kappa)$ per user (parallelizable!)
- Aggregate: $O(n \cdot \kappa)$ $\mathbb{G}_1$ additions
- Verify: $O(n + \kappa)$ pairings

Key observations:

- EncTag dominates (bottleneck)
- Aggregate nearly constant (DLP $O(1)$)
- Verify linear in $n + \kappa$
- EncTag is parallelizable across users

Analysis

Strengths

- **Fast aggregation**
  - DLP table: $O(1)$ lookups
  - Nearly constant in n
- **Public verification**
  - No secrets required
  - Anyone can verify
- **Exact results**
  - No noise (unlike DP)
  - Perfect accuracy
- **Strong security**
  - Privacy: CDH
  - Verifiability: co-CDH

Limitations

- **Encryption cost**
  - $O(\kappa)$ per user
  - Mitigated by parallelization
- **DLP table constraint**
  - $n \times \max < 65536$
  - Limits value range
- **Trusted setup**
  - Data Dealer must be honest
  - Can be distributed via MPC

Conclusion

Implementation

- Complete PUDA protocol in Rust
- All four phases functional
- Benchmarked up to 100 clients
- Using Arkworks ecosystem

What I learned

- Pairing-based cryptography
- Rust for crypto implementation
- Privacy-preserving protocols
- Performance optimization

Possible Extensions

- Distributed key generation (remove trusted setup)
- Larger DLP tables
- GPU acceleration for EncTag
- Integration with FL frameworks
- Robustness to user dropouts

Code Summary

- Language: Rust
- Library: Arkworks (ark-bls12-381)
- Tested: $n \leq 100$, $\kappa \leq 100$

Thank You

Questions?

donval@eurecom.fr